

TP FINAL — PROJET COMPLET

Du dataset au modèle déployé en ligne

Modules 9 & 10 · Pipeline scikit-learn · Interface Streamlit · Déploiement cloud

 Durée	 Modalité	 Fichiers	 Niveau
3h30	Individuel	dataset_assurance_ML.xlsx	Intermédiaire

Objectifs du TP

- Charger le dataset assurance et choisir les variables en évitant les fuites de données
- Construire un Pipeline scikit-learn complet (prétraitement + modèle) et l'évaluer
- Sauvegarder le pipeline avec joblib et l'interroger sur un nouveau client
- Créer une interface web Streamlit pour interroger le modèle avec des curseurs et des menus
- Publier le projet sur GitHub et déployer l'application en ligne, accessible par une URL

Plan du TP

Partie	Contenu	Module	Étapes	Durée
A	Chargement & choix des variables	9.1	1 – 5	30 min
B	Pipeline, entraînement & évaluation	9.1 – 9.2	6 – 11	45 min
C	Sauvegarde & interrogation du modèle	9.3	12 – 15	25 min
D	Interface Streamlit	10.1 – 10.3	16 – 23	60 min
E	GitHub, déploiement en ligne & remise	10.4	24 – 31	50 min

Contexte métier

Vous êtes Data Scientist junior chez un assureur automobile. Le service Fidélisation perd 10 % de ses clients chaque année et souhaite les détecter AVANT qu'ils ne partent. Votre mission : construire un modèle de scoring de résiliation, puis le mettre entre les mains des conseillers sous forme d'une application web où ils saisissent le profil d'un client et obtiennent immédiatement son niveau de risque.

Livrable final

À la fin de ce TP vous aurez : un dépôt GitHub public contenant votre code, un modèle sauvegardé (.pkl) et une application Streamlit accessible à tous via une adresse du type `https://votre-nom-scoring.streamlit.app` — un vrai projet à montrer en entretien.

Mise en place

0.1 Environnement et structure du projet

Ce projet sera publié en ligne : sa structure doit être propre dès le départ. Créez-la exactement comme ci-dessous.

```
# Terminal (Anaconda Prompt ou terminal VS Code)
conda activate formation_ml
pip install streamlit

# Structure du projet
mkdir scoring_resiliation
cd scoring_resiliation
mkdir data
mkdir models
mkdir notebooks
echo > requirements.txt
echo > train_model.py
echo > app.py

# Copier dataset_assurance_ML.xlsx dans data/
# Créer notebooks/tp_final.ipynb et sélectionner le kernel 'formation_ml'

scoring_resiliation/
├── data/
│   └── dataset_assurance_ML.xlsx      ← données brutes
├── models/
│   ├── pipeline_resiliation.pkl      ← créé à l'étape 12
│   └── metadata.json                 ← créé à l'étape 13
├── notebooks/
│   └── tp_final.ipynb                ← exploration (parties A à C)
├── train_model.py                    ← script d'entraînement (partie C)
├── app.py                            ← interface Streamlit (partie D)
├── requirements.txt                  ← dépendances (partie E)
└── README.md                         ← description (partie E)
```

💡 Les parties A à C se font dans le notebook. À l'étape 15, vous regrouperez ce code dans `train_model.py` : un script `.py` se relance en une commande, ce qui est indispensable pour un projet que l'on déploie.

0.2 Le dataset : `dataset_assurance_ML.xlsx`

500 clients × 27 colonnes — le fichier nettoyé au Jour 2. Les noms de colonnes contiennent des accents, des espaces et des symboles (€, %, parenthèses) : recopiez-les exactement.

Groupe	Colonnes	Usage dans ce TP
Identité	N° Police, Nom, Prénom, Sexe, Code Postal	Exclues (aucune valeur prédictive)
Client	Âge, Catégorie Prof., Salaire Annuel (€), Ville	Âge, Salaire, Catégorie Prof.
Contrat	Type Contrat, Statut Contrat, Date Souscription, Date Échéance, Ancienneté (mois), Prime Annuelle (€), Franchise (€), Nb Garanties	Type Contrat, Ancienneté, Prime — ⚠ Statut Contrat = FUITE
Véhicule	Type Véhicule, Usage Véhicule, Puissance Fiscale (CV), Valeur Véhicule (€)	Usage Véhicule
Sinistralité	Coeff. Bonus-Malus, Nb Sinistres (3 ans), Montant Sinistres (€), Dernier Sinistre, Score Risque (0-100)	Toutes (les plus prédictives)
Cible	Résiliation (0 / 1)	Variable à prédire — taux : 10 %

📖 Comment utiliser ce TP

Chaque étape contient une consigne, un squelette de code à compléter (les _____ sont à remplir) et le résultat attendu pour vous auto-vérifier. La correction sera remise par le formateur à la fin de la séance.

PARTIE A — Chargement & choix des variables

Module 9.1 · 30 min · Objectif : préparer X et y sans tricher

ÉTAPE 1 Charger le fichier Excel et le convertir en CSV

- Chargez data/dataset_assurance_ML.xlsx avec pd.read_excel (bibliothèque openpyxl)
- Sauvegardez-le en CSV avec l'encodage utf-8-sig pour la suite du projet
- Rechargez le CSV et vérifiez la shape

```
import pandas as pd, numpy as np
import warnings; warnings.filterwarnings('ignore')

df = pd.read_excel('data/dataset_assurance_ML.xlsx')
df.to_csv('data/dataset_assurance_ML.csv', index=False, encoding='utf-8-sig')

df = pd.read_csv('data/dataset_assurance_ML.csv', encoding='utf-8-sig')
print(df.shape)
```

Résultat attendu : (500, 27). Aucune valeur manquante, aucun doublon (vérifiez avec isnull().sum().sum() et duplicated().sum()).

ÉTAPE 2 La variable cible

- Affichez la répartition de 'Résiliation' en effectifs puis en pourcentage
- Question : le problème est-il équilibré ? Quelle conséquence pour l'entraînement ?

```
TARGET = 'Résiliation'
print(df[TARGET].value_counts())
print(df[TARGET].value_counts(normalize=True).round(2))
```

Résultat attendu : 450 clients restent (0), 50 résilient (1) → 10 %. Classes déséquilibrées : on utilisera class_weight='balanced' et on jugera avec le ROC-AUC, pas l'accuracy.

ÉTAPE 3 Détecter une fuite de données (data leakage)

- Croisez 'Statut Contrat' et 'Résiliation' avec pd.crosstab
- Question : que remarquez-vous ? Peut-on utiliser 'Statut Contrat' pour prédire la résiliation ?

```
print(pd.crosstab(df['Statut Contrat'], df[TARGET]))
```

Résultat attendu : Actif → 450 / 0 · Résilié → 0 / 36 · Suspendu → 0 / 14. Le statut révèle la cible à 100 % : un modèle qui l'utiliserait aurait un score parfait... et serait inutile, puisqu'au moment de prédire on ne connaît pas encore ce statut.

⚠ La fuite de données (data leakage)

Une fuite de données, c'est une variable qui contient (directement ou indirectement) l'information que l'on cherche à prédire, mais qui ne sera PAS disponible au moment où le modèle sera utilisé. Résultat : des scores excellents en test, et un modèle inutile en production. Réflexe à prendre : pour chaque variable, demandez-vous « est-ce que je connaîtrais cette valeur AVANT que le client ne résilie ? ». Si non, elle sort.

ÉTAPE 4 Choisir les variables numériques et catégorielles

- Définissez num_cols (8 variables) et cat_cols (4 variables) selon les listes ci-dessous
- Les variables sont choisies d'après l'EDA du Jour 2 (les plus liées à la cible) et pour garder une interface de saisie raisonnable : 12 champs
- Construisez X et y

```
num_cols = ['Âge', 'Salaire Annuel (€)', 'Prime Annuelle (€)', 'Ancienneté (mois)',
            'Coeff. Bonus-Malus', 'Nb Sinistres (3 ans)',
            'Montant Sinistres (€)', 'Score Risque (0-100)']
```

```
cat_cols = ['Type Contrat', 'Catégorie Prof.', 'Usage Véhicule', 'Dernier Sinistre']

X = df[num_cols + cat_cols]
y = df[TARGET]
print(X.shape, y.shape)
```

Résultat attendu : (500, 12) (500,).

ÉTAPE 5 Vérifier le lien avec la cible

- Calculez la corrélation de chaque variable numérique avec la cible (corrwith) et triez
- Calculez le taux de résiliation par modalité de 'Dernier Sinistre'
- Question : quelles sont les 3 variables numériques les plus liées à la résiliation ?

```
print(X[num_cols].corrwith(y).round(3).sort_values(ascending=False))
print(df.groupby('Dernier Sinistre')[TARGET].mean().round(2).sort_values())
```

Résultat attendu : Score Risque (0,245), Nb Sinistres (0,237), Coeff. Bonus-Malus (0,232). Dernier Sinistre : Vol → 37 % et Bris de glace → 27 % de résiliation, contre 6 % pour Aucun.

PARTIE B — Pipeline, entraînement & évaluation

Modules 9.1 – 9.2 · 45 min · Objectif : un seul objet qui fait TOUT, du brut à la prédiction

Pourquoi un Pipeline ?

Au Jour 2, vous avez appliqué scaler et encodeur « à la main ». Problème : pour prédire sur un nouveau client, il faut refaire exactement les mêmes opérations, dans le même ordre, avec les mêmes objets fittés. Le Pipeline scikit-learn encapsule tout cela : on lui donne des données brutes, il s'occupe du reste. C'est ce qui rendra l'interface de la partie D triviale à écrire.

ÉTAPE 6 Séparer train / test

- Faites un `train_test_split` à 80/20, `random_state=42`, en conservant le ratio de la cible (stratify)
- Vérifiez que le taux de résiliation est bien ~10 % dans les deux jeux

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=____, random_state=42, stratify=____)
print(X_train.shape, X_test.shape)
print(y_train.mean().round(2), y_test.mean().round(2))
```

Résultat attendu : (400, 12) (100, 12) · 0.10 et 0.10.

ÉTAPE 7 Le prétraitement en un seul objet : ColumnTransformer

- Créez un ColumnTransformer qui applique StandardScaler aux `num_cols` et OneHotEncoder aux `cat_cols`
- `handle_unknown='ignore'` : une modalité jamais vue à l'entraînement ne fera pas planter l'application

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), ____),
    ('cat', OneHotEncoder(handle_unknown='ignore'), ____),
])
```

Résultat attendu : Aucune sortie. L'objet sera fitté plus tard, à l'intérieur du pipeline.

ÉTAPE 8 Deux candidats dans un Pipeline

- Créez deux pipelines [prep → model] : une Régression Logistique et un Random Forest
- Les deux avec `class_weight='balanced'` à cause du déséquilibre 90/10
- Pour le Random Forest, limitez la profondeur (`max_depth=4`, `min_samples_leaf=10`) pour éviter le sur-apprentissage sur 400 lignes

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

candidats = {
    'Régression Logistique': LogisticRegression(
        max_iter=1000, class_weight='____', random_state=42),
    'Random Forest': RandomForestClassifier(
        n_estimators=300, max_depth=____, min_samples_leaf=____,
        class_weight='balanced', random_state=42),
}

pipelines = {nom: Pipeline(['prep', preprocessor], ('model', algo))
               for nom, algo in candidats.items() }
```

Résultat attendu : Un dictionnaire de 2 pipelines prêts à être entraînés.

ÉTAPE 9 Comparer par validation croisée

- Pour chaque pipeline, calculez le ROC-AUC en validation croisée 5-fold sur le TRAIN uniquement
- Affichez moyenne $\pm$ écart-type
- Question : lequel retenez-vous ? L'écart est-il significatif ?

```
from sklearn.model_selection import cross_val_score

for nom, pipe in pipelines.items():
    scores = cross_val_score(pipe, X_train, y_train, cv=____, scoring='____')
    print(f'{nom:22s} AUC = {scores.mean():.3f}  $\pm$  {scores.std():.3f}')
```

Résultat attendu : Régression Logistique $\approx 0,622 \pm 0,065$ · Random Forest $\approx 0,626 \pm 0,093$. Écart non significatif : on retient le Random Forest (il fournit en plus les importances de variables, utiles pour l'interface).

💡 *Un AUC de 0,63 est modeste (0,5 = hasard, 1 = parfait). C'est normal : ce jeu de 500 clients contient un signal faible. En entreprise, on améliorerait avec plus de données et de nouvelles variables (historique des réclamations, contacts au service client...). L'objectif de ce TP est la chaîne complète, pas le record de performance.*

ÉTAPE 10 Entraîner le modèle retenu et évaluer sur le test

- Entraînez le pipeline Random Forest sur X_train / y_train
- Prédisez les classes ET les probabilités sur X_test
- Calculez accuracy, F1, ROC-AUC et la matrice de confusion

```
from sklearn.metrics import (accuracy_score, f1_score, roc_auc_score,
                             confusion_matrix, classification_report)

pipeline = pipelines['Random Forest']
pipeline.__(X_train, y_train)

y_pred = pipeline.predict(X_test)
y_proba = pipeline.predict_proba(X_test)[: , ____]

print('Accuracy :', accuracy_score(y_test, y_pred).round(3))
print('F1       :', f1_score(y_test, y_pred).round(3))
print('ROC-AUC  :', roc_auc_score(y_test, y_proba).round(3))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=['Reste', 'Résilie']))
```

Résultat attendu : Accuracy 0,79 · F1 0,323 · AUC 0,649. Matrice : $\begin{bmatrix} 74 & 16 \\ 5 & 5 \end{bmatrix}$ → sur 10 résiliations réelles, le modèle en détecte 5 (recall 50 %), au prix de 16 fausses alertes.

ÉTAPE 11 Lire la matrice de confusion comme un métier

- Répondez dans une cellule Markdown :
- Q1. Un modèle qui prédirait toujours « reste » aurait 90 % d'accuracy. Est-il meilleur que le vôtre ?
- Q2. Pour le service Fidélisation, quelle erreur coûte le plus cher : rater un client qui va partir (faux négatif) ou appeler un client qui serait resté (faux positif) ?
- Q3. Faut-il donc plutôt baisser ou monter le seuil de 0,5 ?

PARTIE C — Sauvegarde & interrogation du modèle

Module 9.3 · 25 min · Objectif : un fichier .pkl + savoir lui poser une question

ÉTAPE 12 Sauvegarder le pipeline complet

- Sauvegardez le pipeline (prétraitement + modèle) dans models/pipeline_resiliation.pkl avec joblib
- Vérifiez la taille du fichier créé

```
import joblib, os
joblib.__(pipeline, 'models/pipeline_resiliation.pkl')
print(os.path.getsize('models/pipeline_resiliation.pkl') / 1024, 'Ko')
```

Résultat attendu : Un fichier d'environ 700 Ko. Il contient le scaler fitté, l'encodeur fitté ET la forêt : un seul fichier à déployer.

ÉTAPE 13 Sauvegarder les métadonnées pour l'interface

- L'interface aura besoin de connaître : le nom des colonnes, les bornes des variables numériques (pour les curseurs) et les modalités des catégorielles (pour les menus)
- Construisez un dictionnaire meta et sauvegardez-le en JSON dans models/metadata.json

```
import json
meta = {
    'modele': 'Random Forest',
    'auc_test': round(float(roc_auc_score(y_test, y_proba)), 3),
    'num_cols': num_cols,
    'cat_cols': cat_cols,
    'num_ranges': {c: {'min': float(X[c].min()), 'max': float(X[c].max()),
                       'median': float(X[c].median())} for c in num_cols},
    'cat_values': {c: sorted(X[c].unique().tolist()) for c in cat_cols},
}
with open('models/metadata.json', 'w', encoding='utf-8') as f:
    json.dump(meta, f, ensure_ascii=__, indent=2)
```

Résultat attendu : Un fichier JSON lisible. Ouvrez-le : Âge va de 18 à 79 (médiane 50), Type Contrat a 4 modalités.

⚠ ensure_ascii=False est indispensable : sans lui, « Âge » serait écrit \u00c2ge dans le fichier et deviendrait illisible.

ÉTAPE 14 Interroger le modèle sur un nouveau client

- Rechargez le pipeline avec joblib.load (dans une NOUVELLE cellule, comme si vous repartiez de zéro)
- Créez un DataFrame d'une ligne décrivant un client à risque (jeune, peu ancien, 3 sinistres, score élevé, contrat Bronze, dernier sinistre Vol)
- Affichez la classe prédite et la probabilité de résiliation
- Modifiez ensuite le profil (0 sinistre, ancienneté 200 mois, Gold, Aucun sinistre) et comparez

```
modele = joblib.load('models/pipeline_resiliation.pkl')

client = pd.DataFrame([
    'Âge': 34, 'Salaire Annuel (€)': 28000, 'Prime Annuelle (€)': 950,
    'Ancienneté (mois)': 6, 'Coeff. Bonus-Malus': 1.25, 'Nb Sinistres (3 ans)': 3,
    'Montant Sinistres (€)': 4200, 'Score Risque (0-100)': 72,
    'Type Contrat': 'Bronze', 'Catégorie Prof.': 'Entrepreneur',
    'Usage Véhicule': 'Professionnel', 'Dernier Sinistre': 'Vol',
])
print('Classe :', modele.__(client))
print('Proba :', modele.__(client)[0, 1].round(3))
```

Résultat attendu : Classe [1], probabilité ≈ 0,668 (67 %). Le profil « fidèle » donne ≈ 0,17. Remarquez : vous avez donné des données BRUTES (texte, euros) — le pipeline a tout transformé seul.

ÉTAPE 15 Provoquer l'erreur classique, puis passer en script

- Supprimez la colonne 'Score Risque (0-100)' du DataFrame client et relancez la prédiction — lisez le message d'erreur
- Question : quelle règle en tirez-vous pour l'interface ?
- Enfin, regroupez tout le code des parties A, B, C (sans les affichages exploratoires) dans train_model.py et exécutez-le depuis le terminal

```
try:
    modele.predict(client.drop(columns=['Score Risque (0-100)']))
except Exception as e:
    print('ERREUR :', e)

# Terminal :
# python train_model.py
```

Résultat attendu : ERREUR : columns are missing: {'Score Risque (0-100)'}. Règle : l'interface doit fournir EXACTEMENT les 12 colonnes, avec les mêmes noms. Le script train_model.py recrée le .pkl et le .json en une commande.

PARTIE D — Interface Streamlit

Modules 10.1 – 10.3 · 60 min · Objectif : une page web pour interroger le modèle, sans HTML ni JavaScript

Streamlit en 30 secondes

Streamlit transforme un script Python en page web. Chaque fonction `st.xxx()` affiche un élément (titre, curseur, bouton, tableau...). À chaque interaction de l'utilisateur, le script est ré-exécuté de haut en bas et la page se met à jour. Vous n'écrivez que du Python.

Fonction	Rôle
<code>st.title()</code> / <code>st.write()</code> / <code>st.caption()</code>	Afficher du texte
<code>st.sidebar.slider(label, min, max, value)</code>	Curseur numérique dans la barre latérale
<code>st.sidebar.selectbox(label, options)</code>	Menu déroulant
<code>st.button(label)</code>	Bouton — renvoie True quand on clique
<code>st.metric(label, valeur)</code>	Gros chiffre mis en avant
<code>st.progress(valeur 0-1)</code>	Barre de progression / jauge
<code>st.success()</code> / <code>st.warning()</code> / <code>st.error()</code>	Message coloré vert / orange / rouge
<code>st.dataframe(df)</code> / <code>st.bar_chart(serie)</code>	Tableau interactif / graphique en barres
<code>@st.cache_resource</code>	Charger le modèle une seule fois, pas à chaque clic

ÉTAPE 16 Première application : Hello Streamlit

- Dans `app.py`, écrivez le code ci-dessous
- Lancez-le depuis le terminal : `streamlit run app.py`
- Le navigateur s'ouvre sur `http://localhost:8501`

```
import streamlit as st

st.title('🚗 Scoring de résiliation – Assurance Auto')
st.write('Bonjour ! Mon premier modèle en ligne.')
```

Résultat attendu : Une page web avec votre titre. Laissez le terminal ouvert : à chaque sauvegarde de `app.py`, cliquez sur « Rerun » (ou activez « Always rerun ») dans le navigateur.

 Si la commande `streamlit` n'est pas reconnue : vérifiez que l'environnement `formation_ml` est activé (son nom entre parenthèses dans le terminal) et que `pip install streamlit` a bien été fait DANS cet environnement.

ÉTAPE 17 Charger le modèle et les métadonnées (une seule fois)

- Ajoutez la fonction `charger_modele()` ci-dessous, décorée avec `@st.cache_resource`
- Configurez la page (titre d'onglet, mise en page large) avec `st.set_page_config`, qui doit être le PREMIER appel Streamlit
- Affichez le nom du modèle et son AUC sous le titre avec `st.caption`

```
import json, joblib
import pandas as pd
import streamlit as st

st.set_page_config(page_title='Scoring Résiliation', page_icon='🚗', layout='wide')

@st._____
def charger_modele():
    pipeline = joblib.load('models/pipeline_resiliation.pkl')
    with open('models/metadata.json', encoding='utf-8') as f:
```

```

        meta = json.load(f)
        return pipeline, meta

pipeline, meta = charger_modele()
num_cols, cat_cols = meta['num_cols'], meta['cat_cols']
rng, cats = meta['num_ranges'], meta['cat_values']

st.title('🚗 Scoring de résiliation – Assurance Auto')
st.caption(f"Modèle : {meta['modele']} · AUC test : {meta['auc_test']}")

```

Résultat attendu : La page affiche « Modèle : Random Forest · AUC test : 0.649 ». Sans le cache, le .pkl serait relu à chaque clic.

ÉTAPE 18 Les curseurs des variables numériques

- Écrivez la fonction `curseur(col, step, fmt)` qui crée un slider dans la barre latérale, borné par le min/max du metadata et positionné sur la médiane
- Créez les 8 curseurs et stockez les valeurs dans un dictionnaire client
- Le format `'%d'` affiche un entier ; renvoyez alors `int(val)`

```

st.sidebar.header('👤 Profil du client')

def curseur(col, step=1.0, fmt=None):
    r = rng[col]
    val = st.sidebar.slider(col, min_value=r['____'], max_value=r['____'],
                            value=r['median'], step=step, format=fmt)
    return int(val) if fmt == '%d' else val

client = {}
client['Âge'] = curseur('Âge', 1.0, '%d')
client['Salaire Annuel (€)'] = curseur('Salaire Annuel (€)', 500.0, '%d')
client['Prime Annuelle (€)'] = curseur('Prime Annuelle (€)', 10.0, '%d')
client['Ancienneté (mois)'] = curseur('Ancienneté (mois)', 1.0, '%d')
client['Coeff. Bonus-Malus'] = curseur('Coeff. Bonus-Malus', 0.01, '%.2f')
client['Nb Sinistres (3 ans)'] = curseur('Nb Sinistres (3 ans)', 1.0, '%d')
client['Montant Sinistres (€)'] = curseur('Montant Sinistres (€)', 100.0, '%d')
client['Score Risque (0-100)'] = curseur('Score Risque (0-100)', 1.0, '%d')

```

Résultat attendu : 8 curseurs dans la barre de gauche, Âge de 18 à 79 positionné sur 50.

ÉTAPE 19 Les menus des variables catégorielles

- Ajoutez un séparateur puis, en boucle sur `cat_cols`, un selectbox par variable alimenté par `cats[col]`
- Question : pourquoi prendre les modalités depuis le metadata plutôt que de les écrire en dur ?

```

st.sidebar.markdown('---')
for col in cat_cols:
    client[col] = st.sidebar.__(col, cats[col])

```

Résultat attendu : 4 menus déroulants. Les modalités viennent du metadata : si on ré-entraîne avec de nouvelles catégories, l'interface se met à jour sans modifier `app.py`.

ÉTAPE 20 Le bouton Prédire et le résultat

- Ajoutez un bouton « Prédire » ; dans le bloc if, construisez un DataFrame d'une ligne à partir de client, dans l'ordre `num_cols + cat_cols`
- Calculez la probabilité, affichez-la avec `st.metric`, une jauge `st.progress`, et un message coloré selon deux seuils
- Ajoutez un `st.info()` dans le else pour guider l'utilisateur avant le premier clic

```

SEUIL_RISQUE = 0.55
SEUIL_MODERE = 0.40

if st.button('🔮 Prédire', type='primary', use_container_width=True):

```

```

df_client = pd.DataFrame([client])[____ + ____]
proba = float(pipeline.predict_proba(df_client)[0, ____])

coll, col2 = st.columns([1, 2])
with coll:
    st.metric('Probabilité de résiliation', f'{proba:.0%}')
    if proba >= SEUIL_RISQUE:
        st.error('⚠ Client À RISQUE – action de rétention conseillée')
    elif proba >= SEUIL_MODERE:
        st.warning('⚡ Risque modéré – à surveiller')
    else:
        st.success('✅ Client fidèle – risque faible')
with col2:
    st.write('Niveau de risque')
    st.progress(proba)
    st.write('Données envoyées au modèle :')
    st.dataframe(df_client.T.astype(str).rename(columns={0: 'Valeur'}),
                  use_container_width=True)
else:
    st.info('💡 Ajustez le profil dans la barre latérale, puis cliquez sur Prédire.')

```

Résultat attendu : Profil médian (tous les curseurs par défaut) → 39 %, « Client fidèle ». Poussez Bonus-Malus, Nb Sinistres, Montant et Score au maximum, Ancienneté au minimum → 64 %, « Client À RISQUE ».

⚠ `.astype(str)` est nécessaire : une colonne qui mélange nombres et textes (50 et « Bronze ») fait planter l'affichage du tableau.

Profil du client

Âge

50

Salaire Annuel (€)

36117

Prime Annuelle (€)

693

Ancienneté (mois)

128

Coeff. Bonus-Malus

0.80

Nb Sinistres (3 ans)

0

Montant Sinistres (€)

0

Score Risque (0-100)

13

Scoring de résiliation — Assurance Auto

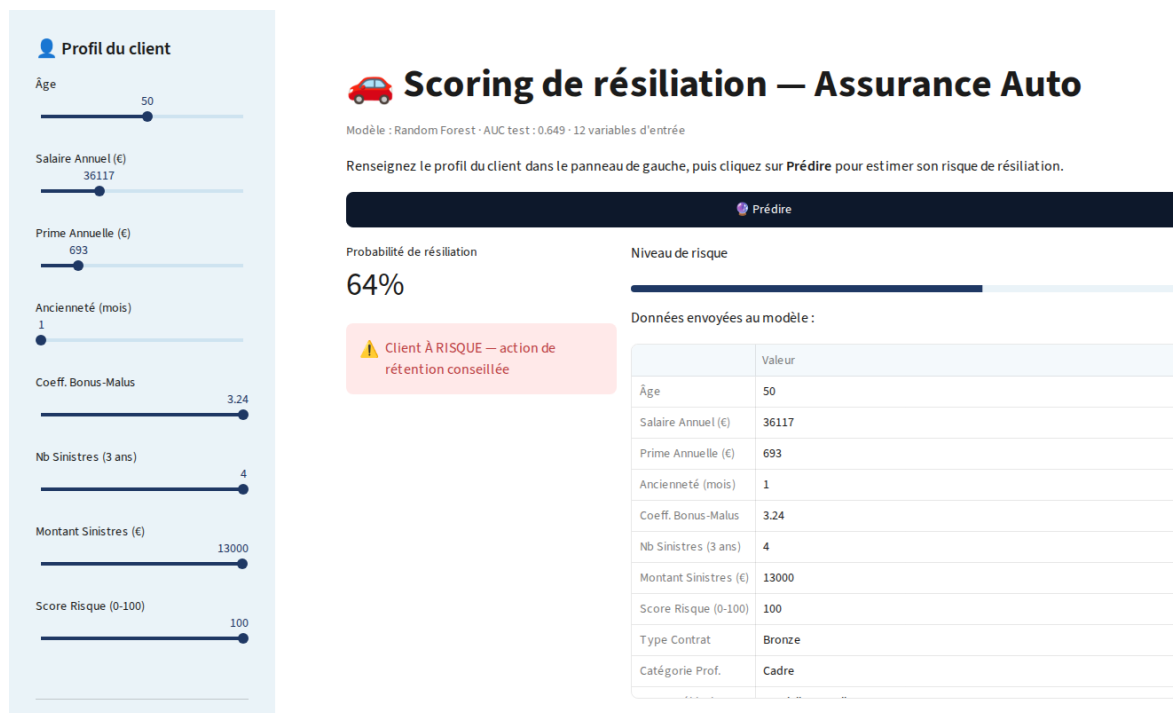
Modèle : Random Forest · AUC test : 0.649 · 12 variables d'entrée

Renseignez le profil du client dans le panneau de gauche, puis cliquez sur **Prédire** pour estimer son risque de résiliation.

Prédire

Ajustez le profil dans la barre latérale, puis cliquez sur Prédire.

Écran d'accueil : curseurs et menus à gauche, bouton Prédire au centre



Après un clic sur Prédire avec un profil à risque : 64 %, alerte rouge, jauge et récapitulatif

ÉTAPE 21 Expliquer la prédiction : importance des variables

- Toujours dans le bloc if, récupérez le Random Forest avec `pipeline.named_steps['model']` et les noms de colonnes transformées avec `pipeline.named_steps['prep'].get_feature_names_out()`
- Construisez une Series des 8 importances les plus fortes et affichez-la avec `st.bar_chart`
- Les noms sont préfixés 'num__' ou 'cat__' : nettoyez-les avec `split('__', 1)[1]`

```
model = pipeline.named_steps['model']
if hasattr(model, 'feature_importances_'):
    noms = pipeline.named_steps['prep'].get_feature_names_out()
    imp = (pd.Series(model.feature_importances_, index=noms)
           .sort_values(ascending=False).head(8))
    imp.index = [n.split('__', 1)[1] for n in imp.index]
    st.subheader('📊 Les 8 variables les plus influentes du modèle')
    st.bar_chart(imp)
```

Résultat attendu : Un graphique en barres sous le résultat. Coeff. Bonus-Malus ($\approx 0,14$), Salaire et Score Risque ($\approx 0,11$) en tête.

ÉTAPE 22 Personnaliser le thème

- Créez le dossier `.streamlit/` à la racine du projet et le fichier `config.toml` ci-dessous
- Relancez `streamlit run app.py` : le bouton et les curseurs passent en bleu marine

```
# .streamlit/config.toml
[theme]
primaryColor = "#1F3864"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#EAF3F8"
textColor = "#1B1B1B"
```

Résultat attendu : Une interface aux couleurs de la formation. Ce fichier est versionné avec le projet : le thème sera identique en ligne.

ÉTAPE 23 Tester comme un conseiller

- Testez 3 profils et notez la probabilité : (a) profil médian, (b) jeune conducteur Bronze avec 2 sinistres et un Vol, (c) client Gold ancien sans sinistre
- Question : les résultats sont-ils cohérents avec les corrélations de l'étape 5 ?

- Question : que se passe-t-il si vous changez uniquement la Ville dans votre esprit ? (indice : elle n'est pas dans le modèle — pourquoi ?)

PARTIE E — GitHub & déploiement en ligne

Module 10.4 · 50 min · Objectif : une URL publique que n'importe qui peut ouvrir

🔗 Où déployer une application Streamlit ?

Une application Streamlit est un serveur Python qui tourne en permanence et garde une connexion ouverte avec le navigateur (WebSocket). Elle a donc besoin d'un hébergeur capable de faire tourner un processus Python long. Le choix naturel — gratuit, officiel, sans configuration — est Streamlit Community Cloud : on connecte son compte GitHub, on désigne le dépôt, et l'application est en ligne en 2 minutes.

Hébergeur	Streamlit ?	Gratuit ?	Remarque
Streamlit Community Cloud	☑ Natif	Oui	Notre choix — share.streamlit.io
Hugging Face Spaces	☑ Natif	Oui	Bonne alternative, même principe
Render / Railway	☑ Via Docker ou commande	Limité	Plus technique
Vercel	✗ Non adapté	Oui	Voir encadré ci-dessous

? Et Vercel ?

Vercel est conçu pour les sites statiques et les fonctions « serverless » : chaque requête démarre une fonction, qui répond puis s'éteint en quelques secondes. Streamlit fonctionne à l'inverse (processus permanent + WebSocket) : il ne peut pas tourner sur Vercel. Pour déployer sur Vercel, il faudrait remplacer Streamlit par une API Flask ou FastAPI (une fonction /predict qui reçoit du JSON et renvoie la probabilité) et écrire la page web à part. C'est une excellente suite pour aller plus loin — voir le Défi bonus — mais ce n'est pas l'outil adapté à une première mise en ligne.

ÉTAPE 24 Figer les dépendances : requirements.txt

- Le serveur distant doit installer EXACTEMENT les mêmes versions que vous, sinon le .pkl peut refuser de se charger
- Notez vos versions avec `pip show`, puis écrivez `requirements.txt`

```
# Terminal - relever les versions installées
pip show streamlit scikit-learn pandas joblib openpyxl | findstr "Name Version"

# requirements.txt (adaptez aux versions affichées)
streamlit==1.63.0
scikit-learn==1.8.0
pandas==3.0.2
joblib==1.5.3
openpyxl==3.1.5
```

Résultat attendu : Un fichier de 5 lignes. La version de scikit-learn est la plus critique : un .pkl créé avec 1.8 chargé avec 1.5 provoque des erreurs.

ÉTAPE 25 Préparer le dépôt : .gitignore et README.md

- Créez .gitignore pour exclure les fichiers inutiles du dépôt
- Créez README.md : titre, une phrase de description, structure du projet, commande de lancement
- ATTENTION : `models/` et `data/` doivent être inclus — l'application en ligne a besoin du .pkl et du .json

```
# .gitignore
__pycache__/
*.pyc
.env
venv/
.ipynb_checkpoints/
```

```
*.log
```

Résultat attendu : Un dépôt propre : code, modèle, données, dépendances, documentation. Pas de caches ni de secrets.

ÉTAPE 26 Créer le dépôt GitHub

- Créez un compte sur <https://github.com> si nécessaire
- Bouton « New repository » : nom `scoring-resiliation`, Public, sans README (vous avez le vôtre), puis Create
- GitHub affiche des commandes : ne les tapez pas encore, passez à l'étape 27

Résultat attendu : Une page « Quick setup » avec l'URL `https://github.com/VOTRE_COMPTE/scoring-resiliation.git`

ÉTAPE 27 Envoyer le projet sur GitHub

- Installez Git si nécessaire (<https://git-scm.com>), puis configurez votre identité une fois pour toutes
- Dans le terminal, à la racine de `scoring_resiliation/`, exécutez les commandes ci-dessous
- Rafraîchissez la page GitHub : vos fichiers apparaissent

```
git config --global user.name "Votre Nom"
git config --global user.email "vous@exemple.com"

git init
git add .
git commit -m "Scoring résiliation : pipeline + app Streamlit"
git branch -M main
git remote add origin https://github.com/VOTRE_COMPTE/scoring-resiliation.git
git push -u origin main
```

Résultat attendu : Le push demande vos identifiants GitHub (mot de passe = un « Personal Access Token » à créer dans Settings → Developer settings → Tokens). Le dépôt en ligne contient `app.py`, `train_model.py`, `models/`, `data/`, `requirements.txt`, `README.md`, `.streamlit/`.

⚠ Vérifiez sur GitHub que `models/pipeline_resiliation.pkl` est bien présent. Si le fichier manque, l'application en ligne plantera avec `FileNotFoundException`.

ÉTAPE 28 Déployer sur Streamlit Community Cloud

- Allez sur <https://share.streamlit.io> et connectez-vous avec votre compte GitHub (autorisez l'accès)
- Cliquez « Create app » → « Deploy a public app from GitHub »
- Repository : `VOTRE_COMPTE/scoring-resiliation` · Branch : `main` · Main file path : `app.py`
- App URL : choisissez un nom (ex. `votre-nom-scoring`)
- Advanced settings → Python version : choisissez la version proche de la vôtre (`python --version`)
- Cliquez « Deploy » et observez le journal d'installation

Résultat attendu : Après 1 à 3 minutes (« Your app is in the oven »), l'application s'ouvre sur `https://votre-nom-scoring.streamlit.app` — identique à votre version locale.

ÉTAPE 29 Tester en ligne et partager

- Ouvrez l'URL sur votre téléphone : la barre latérale se replie derrière une flèche en haut à gauche
- Faites une prédiction avec le profil « à risque » de l'étape 20 : vous devez retrouver 64 %
- Envoyez l'URL à un binôme et demandez-lui une prédiction : vous venez de livrer un modèle en production

Résultat attendu : Même résultat qu'en local. Si l'application affiche une erreur, cliquez « Manage app » en bas à droite pour lire le journal.

ÉTAPE 30 Le cycle de mise à jour

- Modifiez `app.py` : par exemple, changez `SEUIL_RISQUE` à 0.50
- Committez et poussez : `git add . / git commit -m "seuil 0.50" / git push`

- Rechargez l'application en ligne (ou attendez : Streamlit Cloud redéploie automatiquement à chaque push)

```
git add .
git commit -m "Ajustement du seuil de risque à 0.50"
git push
```

Résultat attendu : L'application en ligne se met à jour en moins d'une minute. C'est le principe du déploiement continu : GitHub est la source de vérité.

ÉTAPE 31 Remettre votre travail

- Envoyez un e-mail à loic.ebanga@orange.com avec pour objet : [Formation ML] TP final — NOM Prénom
- Dans le corps : le lien de votre application déployée (<https://....streamlit.app>), le lien de votre dépôt GitHub, et la probabilité obtenue pour le profil « à risque » de l'étape 20
- Joignez une capture d'écran de l'application en ligne avec une prédiction affichée

Objet : [Formation ML] TP final — DUPONT Marie

Bonjour,

Voici mon projet de scoring de résiliation :

- Application en ligne : <https://marie-dupont-scoring.streamlit.app>
- Dépôt GitHub : <https://github.com/marie-dupont/scoring-resiliation>
- Profil à risque (étape 20) : 64 %

Cordialement,
Marie Dupont

Résultat attendu : Un accusé de réception du formateur. Le TP est validé lorsque le lien s'ouvre et qu'une prédiction fonctionne.

🔍 Dépannage — les 5 erreurs les plus fréquentes au déploiement

Message / symptôme	Cause	Solution
ModuleNotFoundError: No module named 'sklearn'	requirements.txt absent ou incomplet	Ajouter scikit-learn et redéploier (Manage app → Reboot)
FileNotFoundError: models/pipeline_resiliation.pkl	Le .pkl n'est pas dans le dépôt	Vérifier .gitignore, git add models/, push
InconsistentVersionWarning / erreur au joblib.load	Version scikit-learn différente	Figurer la même version dans requirements.txt
L'app tourne en local mais pas en ligne : chemin introuvable	Chemin absolu C:/Users/... dans le code	Utiliser des chemins relatifs : 'models/...'
Installation qui échoue sur pandas / numpy	Version Python du cloud trop récente ou trop ancienne	Advanced settings → choisir la même version que la vôtre

Défi bonus, grille d'évaluation & aide-mémoire

🔧 DÉFI BONUS (si vous avez fini en avance)

- Ajoutez un curseur « Seuil d'alerte » (0,3 à 0,7) dans la barre latérale pour que le conseiller règle lui-même la sensibilité du modèle
- Ajoutez un onglet « Scoring par lot » (st.tabs) : st.file_uploader accepte un CSV de plusieurs clients, le modèle score chaque ligne, et st.download_button renvoie le fichier enrichi d'une colonne Probabilité
- Affichez, sous la jauge, le pourcentage de clients du dataset ayant une probabilité inférieure à celle du client saisi (« ce client est plus risqué que 82 % du portefeuille »)
- Version API : écrivez api.py avec FastAPI et une route POST /predict qui reçoit le JSON d'un client et renvoie la probabilité, testez-la avec <http://localhost:8000/docs> — c'est cette version qui pourrait être hébergée sur Vercel

Grille d'évaluation (20 points)

Partie	Critères	Points
A — Variables	Chargement, cible analysée, fuite de données identifiée et exclue, X / y corrects	3
B — Pipeline	ColumnTransformer + Pipeline, split stratifié, CV, métriques adaptées au déséquilibre, lecture métier	5
C — Sauvegarde	.pkl et metadata.json créés, prédiction sur un client brut, train_model.py exécutable	3
D — Interface	Curseurs et menus alimentés par le metadata, prédiction correcte, messages par seuil, importances	5
E — Déploiement	Dépôt GitHub propre (requirements figé, README), application en ligne fonctionnelle, lien envoyé au formateur	4

Aide-mémoire des commandes utilisées

scikit-learn	Streamlit	Git & déploiement
ColumnTransformer([['num', scaler, cols), ...])	st.set_page_config(layout='wide')	git init / git add . / git commit -m "..."
Pipeline([['prep', prep), ('model', algo)])	@st.cache_resource	git branch -M main
cross_val_score(pipe, X, y, cv=5, scoring='roc_auc')	st.sidebar.slider(label, min, max, value)	git remote add origin URL
pipe.fit(X_train, y_train)	st.sidebar.selectbox(label, options)	git push -u origin main
pipe.predict_proba(X)[:, 1]	st.button(label, type='primary')	pip show paquet → requirements.txt
confusion_matrix / roc_auc_score / f1_score	st.metric / st.progress / st.error	share.streamlit.io → Create app
joblib.dump(pipe, 'x.pkl') / joblib.load('x.pkl')	st.dataframe / st.bar_chart	streamlit run app.py
pipe.named_steps['model'].feature_importances_	pipe.named_steps['prep'].get_feature_names_out()	Manage app → logs / Reboot

🏠 Ce que vous avez accompli

En trois jours vous êtes passé d'un terminal vide à une application de Machine Learning en ligne : environnement et projet (Jour 1), exploration et nettoyage (Jour 2), modélisation, évaluation, interface et déploiement (Jour 3). Le dépôt GitHub et l'URL de ce TP sont vos premières pièces de portfolio : ajoutez-les à votre CV et à votre profil LinkedIn.

—— **Bon TP et bonne mise en ligne !** ——